

Explicacao doCodigo

Secador Regenerativo (ESP32)

Arquivo: src/Secador_Novo2026.cpp

Guia comentado, linha a linha, das principais logicas do programa

1. Visao geral do programa

O programa controla um secador regenerativo com 3 estagios de resistencia + 1 resistencia auxiliar, le sensores de umidade/temperatura (AHT20) e de pressao/temperatura (BMP180), mostra tudo num display LCD e detecta o fluxo de ar por uma chave mecanica. A ideia central e simples: cada estagio liga a resistencia quando o sensor do TOPO detecta umidade alta (ar entrando umido) e desliga quando o sensor da BASE indica que o ar ja secou. Existem tambem travas de seguranca (temperatura maxima e tempo maximo de aquecimento) que sobrepoe essa logica normal.

2. Pinos de entrada/saida

Constantes que mapeiam cada pino fisico da ESP32-S3 para uma funcao do sistema:

```
const int ventilador01 = 10;          // liga/desliga o ventilador unico
const int chaveFluxo = 18;            // entrada digital da chave mecanica de fluxo
const int resistenciaEstagio1..3 = ..; // saidas que ligam cada resistencia de estagio
const int resistenciaAuxiliar = 11;    // saida da resistencia auxiliar (caixa/frio)
const int botao01..04 = ..;           // entradas dos 4 botoes do painel
const int ledResistencia1/2 = ..;     // saidas dos LEDs indicadores
```

As macros BTN1..BTN4 so existem para deixar o loop() mais curto: cada uma e um digitalRead() do botao correspondente, ja que os botoes usam pull-up interno (pressionado = LOW).

3. Sensores e organizacao dos dados

O sistema tem 7 sensores AHT20 (umidade + temperatura), todos no mesmo endereco I2C (0x38), por isso passam por um multiplexador TCA9548A: cada sensor fica num canal diferente (0 a 6) e o programa troca de canal antes de falar com cada um. O sensor BMP180 (pressao/temperatura do topo) fica no canal 7 do mesmo multiplexador.

```
#define sens_cima_est1 0    // indices nomeados: cada posicao do array de 7
#define sens_baixo_est1 1  // sensores representa um ponto fisico (topo/base
...                          // de cada estagio) ou a caixa de controle
#define sens_caixa 6

const int cima_estagio[3] = {sens_cima_est1, sens_cima_est2, sens_cima_est3};
const int baixo_estagio[3] = {sens_baixo_est1, sens_baixo_est2, sens_baixo_est3};
// esses dois arrays existem so pra permitir percorrer os 3 estagios num
// for(i=0..2) em vez de repetir o mesmo bloco de codigo 3 vezes
```

umidadeAHT[7] e temperaturaAHT[7] guardam a ultima leitura de cada sensor. umidadeCaixa/temperaturaCaixa sao apenas referencias (apelidos) para a posicao 6 desses arrays, so para o codigo ficar mais legivel quando fala da caixa de controle.

Limites de controle (defines)

```
#define LIMITE_LIGAR_UMIDADE 80      // umidade no TOPO que dispara aquecimento
#define LIMITE_DESLIGAR_UMIDADE 72  // umidade na BASE que confirma que secou
#define TEMPERATURA_SEGURANCA_MAX 90.0 // corte de seguranca por temperatura
#define TEMPO_MAX_AQUECIMENTO_MS (30*60*1000) // corte por tempo (30 min)
#define TEMPERATURA_FRIO_EXTREMO 0.0 // abaixo disso liga a resist. auxiliar
#define UMIDADE_CAIXA_MAX 80         // acima disso liga por anti-condensacao
#define LIMITE_DIRECAO_PA 5.0        // variacao minima p/ considerar direcao
```

Esses numeros sao os "botoes de ajuste" do sistema: mudar o comportamento do secador na pratica normalmente significa mexer so nesses valores.

Arrays de funcao (funcLigaResistencia / funcDesligaResistencia)

```
void (*funcLigaResistencia[3])() = { ligaResistenciaEstagio1, ligaResistenciaEstagio2,
ligaResistenciaEstagio3 };
void (*funcDesligaResistencia[3])() = { desligaResistenciaEstagio1, ... };
```

Em vez de escrever "if (i==0) ligaResistenciaEstagio1(); else if (i==1) ..." dentro do loop de controle, o codigo guarda um array com o "endereco" de cada funcao. Assim, dentro do for(i=0..2), basta chamar funcLigaResistencia[i]() para ligar o estagio certo - a mesma logica do controlarEstagios() serve para os 3 estagios sem repetir codigo.

4. setup() - inicializacao

Roda uma vez quando a ESP32 liga. Ordem importa: o barramento I2C precisa estar pronto antes de qualquer sensor, e os sensores precisam estar prontos antes da primeira leitura/exibicao.

```
Wire.begin(40, 41);      // inicia o barramento I2C nos pinos SDA=40, SCL=41
Wire.setTimeout(50);     // limita a espera do I2C p/ nao travar o watchdog
                        // se um canal do multiplexador estiver sem sensor
lcd.init(); lcd.backlight(); // liga o display
inicializarNVS();         // prepara a memoria persistente (flash)
configurarPinos();        // define cada pino como entrada ou saida
inicializarSensores();    // detecta/inicializa BMP180 e os 7 AHT20
inicializarWatchdog();    // liga o watchdog (reinicia a ESP32 se o codigo travar)
lerSensores(); exibirLCD(); // primeira leitura e primeira tela, antes do loop comecar
```

5. loop() - programa principal

Executa sem parar. E o "maestro": le sensores, atualiza a tela, olha os botoes e decide se liga/desliga cada resistencia.

```
lerSensores();           // atualiza temperatura/umidade de todos os sensores
lerChaveFluxo();          // atualiza fluxoAtivo (com debounce)
exibirLCD();              // redesenha a tela (no maximo 1x/segundo, ver secao 8)

sistemaTravado = fluxoAtivo; // trava geral do sistema segue a chave de fluxo

if (BTN1 == 0) menu = 1;   // cada botao 1-3 so troca a tela mostrada no LCD
if (BTN2 == 0) menu = 2;
if (BTN3 == 0) menu = 3;
if (BTN4 == 0) {           // botao 4 e um atalho: forca ligar o
    funcLigaResistencia[0](); // estagio 1 na mao, sem esperar a
    estagioAquecendo[0] = true; // condicao normal de umidade
    tempoInicioAquecimento[0] = millis();
}

controlarEstagios();       // logica automatica de liga/desliga dos 3 estagios
controlarAuxiliar();       // logica da resistencia auxiliar (frio/caixa)

esp_task_wdt_reset();      // avisa o watchdog que o loop nao travou
```

6. lerChaveFluxo() - deteccao de fluxo de ar (debounce)

A chave mecanica pode "tremar" no momento da transicao (ligar/desligar), gerando leituras falsas rapidas. O debounce resolve isso exigindo que a leitura fique estavel por um tempo minimo antes de acreditar nela.

```
bool leituraAtual = digitalRead(chaveFluxo);

if (leituraAtual != leituraFluxoAnterior) {    // mudou agora mesmo?
    tempoUltimaMudancaFluxo = millis();        // reinicia a contagem do debounce
    leituraFluxoAnterior = leituraAtual;
}

if ((millis() - tempoUltimaMudancaFluxo) > DEBOUNCE_FLUXO) { // ficou estavel
    fluxoAtivo = (leituraAtual == LOW);    // tempo suficiente -> confirma o estado
}                                           // (LOW = chave fechada = ar passando)
```

Ou seja: toda vez que o pino muda de valor, o relógio do debounce reinicia. So depois de 300ms (DEBOUNCE_FLUXO) sem nenhuma mudanca e que o programa realmente confia na leitura e atualiza fluxoAtivo.

7. controlarEstagios() - o coracao do controle

Roda um for(i=0..2) que repete a MESMA logica para os 3 estagios (por isso nao ha 3 blocos de codigo iguais). Para cada estagio, decide na seguinte ordem de prioridade:

- 1) Seguranca: se o sistema esta travado, se a temperatura do topo passou de TEMPERATURA_SEGURANCA_MAX, ou se o tempo ligado passou de TEMPO_MAX_AQUECIMENTO_MS -> forca desligar, nao importa o resto.
- 2) Se nao esta aquecendo: liga quando a umidade do TOPO desse estagio ultrapassa LIMITE_LIGAR_UMIDADE (chegou ar umido).
- 3) Se ja esta aquecendo: desliga quando a umidade da BASE cai abaixo de LIMITE_DESLIGAR_UMIDADE (ar ja secou o suficiente).

```
if (sistemaTravado) forcarDesligar = true;                // seguranca 0
else if (temperaturaAHT[indiceTopo] > TEMPERATURA_SEGURANCA_MAX) forcarDesligar = true; // seguranca 1
else if (estagioAquecendo[i] && (millis()-tempoInicioAquecimento[i] > TEMPO_MAX_AQUECIMENTO_MS))
    forcarDesligar = true;                                // seguranca 2

if (forcarDesligar) { ...desliga... }
else if (!estagioAquecendo[i]) {                          // estava desligado
    if (umidadeAHT[indiceTopo] > LIMITE_LIGAR_UMIDADE) ...liga...
} else {                                                    // estava ligado
    if (umidadeAHT[indiceBase] < LIMITE_DESLIGAR_UMIDADE) ...desliga...
}
```

Depois do for, o codigo acende os LEDs conforme estagioAquecendo[0]/[1] (so ha 2 LEDs fisicos hoje) e decide o ventilador: ele so liga se ALGUM estagio estiver aquecendo E o sistema nao estiver travado - o ventilador nunca deve rodar se a chave de fluxo indicar que o ar nao pode circular.

8. controlarAuxiliar() - resistencia auxiliar

Protege contra dois cenários independentes, ligando a mesma resistência auxiliar para os dois:

- Frio extremo na entrada (`temperaturaEntrada < TEMPERATURA_FRIO_EXTREMO`).
- Umidade excessiva dentro da caixa de controle (anti-condensacao), quando `umidadeCaixa > UMIDADE_CAIXA_MAX`.

```
bool precisaAquecer = (temperaturaEntrada < TEMPERATURA_FRIO_EXTREMO)
                    || (umidadeCaixa > UMIDADE_CAIXA_MAX);

if (precisaAquecer && !auxiliarAquecendo) { ...liga... } // so age na
else if (!precisaAquecer && auxiliarAquecendo) { ...desliga... } // transicao
```

O uso de "`&& !auxiliarAquecendo`" / "`&& auxiliarAquecendo`" evita chamar `liga()/desliga()` e imprimir no Serial repetidamente a cada volta do loop - so age quando o estado realmente muda.

9. lerBMP180Topo() - temperatura, pressao e direcao do fluxo

Le o sensor de pressao do topo em duas etapas (medir temperatura, depois medir pressao), cada uma com um pequeno timeout de 5s esperando o sensor terminar a conversao. Se o sensor nao responder em qualquer etapa, a funcao retorna false e so loga um erro - isso NAO impede a leitura dos outros sensores AHT20, que segue normalmente em lerSensores().

```
pressaoTopoAnterior = pressaoTopoAtual; // guarda a leitura antiga antes
pressaoTopoAtual = bmp180Topo.getPressure(); // de sobrescrever, p/ poder comparar

float deltaPressaoTopo = pressaoTopoAtual - pressaoTopoAnterior;
if (deltaPressaoTopo > LIMIAR_DIRECAO_PA) direcaoFluxo = FLUXO_EXPELINDO; // pressao subindo
else if (deltaPressaoTopo < -LIMIAR_DIRECAO_PA) direcaoFluxo = FLUXO_PUXANDO; // pressao caindo
else direcaoFluxo = FLUXO_ESTAVEL; // variacao pequena
```

A logica e simples: compara a pressao atual com a anterior. Se a diferenca (delta) for maior que o limiar, o ar esta sendo "expelido"; se for menor (mais negativa que o limiar), esta sendo "puxado"; caso contrario, considera-se estavel. Esse valor e usado hoje so como informacao/log, exibido no LCD (menu 2) e no Serial.

10. lerSensores() - leitura de todos os sensores

Chama lerBMP180Topo() e depois percorre os 7 sensores AHT20 num unico for, repetindo a mesma logica para cada canal do multiplexador: seleciona o canal (configurarTCA9548A), confirma que o sensor esta disponivel e tem dado novo (available()), e so entao le temperatura e umidade. No final, imprime tudo no Serial para debug/acompanhamento.

11. configurarTCA9548A() / i2cDispositivoPresente()

O TCA9548A e um "multiplexador" I2C: como os 7 sensores AHT20 tem o mesmo endereco (0x38), so pode existir um por vez no barramento. configurarTCA9548A(canal) manda pro multiplexador (endereco 0x70) um byte com um unico bit ligado ($1 \ll \text{canal}$), que abre so aquele canal. i2cDispositivoPresente() apenas testa se algum dispositivo responde num endereco, sem chamar a biblioteca do sensor - isso evita travar o driver I2C tentando iniciar um sensor que fisicamente nao esta conectado naquele canal.

12. exibirLCD() - o que aparece no display

Atualiza a tela no maximo 1 vez por segundo (ou imediatamente se 'ativado' foi setado). Quando o menu muda, limpa a tela antes de redesenhar, para nao sobrar texto da tela anterior. Existem 3 menus, trocados pelos botoes 1/2/3:

- Menu 1: umidade topo/base de cada estagio + '*' indicando qual esta aquecendo, e umidade/temperatura da caixa de controle.
- Menu 2: temperatura ambiente, pressao e direcao do fluxo (dados do BMP180), e se o sistema esta travado.
- Menu 3: status geral - cada estagio ON/OFF, auxiliar ON/OFF e ventilador ON/OFF (calculado com a mesma condicao usada em controlarEstagios(), so pra exibir - nao duplica a decisao de controle).

Os 3 blocos de cada menu repetem a mesma ideia (setCursor + print), por isso so o menu 1 e detalhado abaixo como exemplo:

```
lcd.print("E1 T:");  
dtostrf(umidadeAHT[sens_cima_est1], 3, 0, textoLCD); lcd.print(textoLCD); // umidade do topo  
lcd.print("% B:");  
dtostrf(umidadeAHT[sens_baixo_est1], 3, 0, textoLCD); lcd.print(textoLCD); // umidade da base  
lcd.print(estagioAquecendo[0] ? '*' : ' '); // marca visual: estagio aquecendo ou nao
```

dtostrf() formata o numero com largura fixa (ex: sempre 3 digitos). Isso evita que, ao trocar um valor grande ("100") por um pequeno ("9"), sobre um digito antigo na tela sem ser apagado.

13. salvarDados() / lerDados() - memoria persistente (NVS)

Guardam/leem um valor booleano na memoria flash da ESP32 (sobrevive a desligamentos), usando o namespace "storage". salvarDados() abre a NVS em modo escrita, grava o valor com nvs_set_u8() e confirma com nvs_commit(). lerDados() abre em modo leitura e usa nvs_get_u8(); se a chave nao existir ou der erro, retorna false por padrao. Hoje essas funcoes existem como utilitario generico (ainda nao sao chamadas em nenhum outro ponto do codigo). O handle da NVS e guardado na variavel identificadorNVS, e o status de cada operacao fica em resultado / resultadoLeitura.

14. inicializarWatchdog() e o esp_task_wdt_reset()

O watchdog e uma protecao: se o loop() travar (por exemplo, um sensor que nao responde e prende o codigo num loop infinito), a ESP32 e reiniciada automaticamente apos TEMPO_LIMITE_WATCHDOG (30s) sem receber o "reset". Por isso esp_task_wdt_reset() e chamado no final de cada volta do loop() - e como avisar "ainda estou vivo". A inicializacao tenta a API nova (struct de configuracao); se a build do core nao aceitar, cai para a assinatura antiga (timeout em segundos + bool de panico) como fallback.